C3 Unifying datasets

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 2
C3 Unifying datasets 3
C3-L1 OneLake shortcuts, and the delete that goes through 3
C3-L2 Mirroring operational databases into OneLake 7
C3-L3 Mirroring Azure Databricks Unity Catalog into Fabric 11
Track C best-practice checklist 16
Appendix: solutions and answer keys 17

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C3 Unifying datasets

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Shortcuts, mirroring (including Azure Databricks Unity Catalog) and avoiding duplicate copies.

C3-L1 OneLake shortcuts, and the delete that goes through

Concept

A **shortcut** is a folder in a lakehouse (or KQL database) that points to data somewhere else. It behaves like a symbolic link. The target can be inside OneLake, such as a table in another workspace, or outside it: ADLS Gen2, Azure Blob Storage, Amazon S3 and S3-compatible storage, Google Cloud Storage, Dataverse, OneDrive or SharePoint.

Shortcuts let every Fabric engine read the data **without copying it**. OneLake handles the connection and credentials, so each workload does not need its own.

Where you create the shortcut matters:

- Under **Tables**, point it at Delta data. The lakehouse recognises the folder as a table, and the SQL analytics endpoint and Direct Lake can use it.
- Under **Files**, it can point at data in any format.

Deletes are the trap. Microsoft Learn states the rule plainly:

- Deleting the **shortcut itself** deletes only the shortcut. The target data stays.
- Deleting a **file or folder inside** the shortcut deletes it **in the target**, if your identity has write permission there.

So a clean-up notebook that runs `rm -r` on `Files/partner_feed/2025/` removes the partner's 2025 files from their storage account, not a local copy.

Other points from Learn (checked September 2026):

- Moving, renaming or deleting the target breaks the shortcut. Broken shortcuts appear in the Unidentified area.
- For external shortcuts you can turn on **caching**. OneLake keeps files it has read for 1 to 28 days, which cuts cross-cloud egress costs.

- With Direct Lake over SQL or T-SQL in delegated identity mode, the calling user's identity is not passed through to the target. Check which identity reaches the source before relying on its permissions.

Why it matters

A lakehouse held a shortcut `Files/finance_share` to a folder in the finance team's storage account. An analyst ran a notebook to free up space: it deleted every subfolder older than two years under `Files/`. Their identity had write access to the finance account for another project. Seven years of finance exports disappeared from the source. The storage account had no soft delete, and the files were gone.

Worked example: weak code

```
1 for path in notebookutils.fs.ls("Files/"):
2     if path.name.startswith("2023"):
2         notebookutils.fs.rm(path.path, True)
```

1. **Line 1.** Walks every folder under Files, shortcuts included.
2. **Line 3.** Recursive delete inside a shortcut deletes the target data, if the identity can write there.

Worked example: strong code

```
1 OWNED_PREFIXES = ("Files/landing/", "Files/tmp/")
2
3
4 def safe_to_delete(path: str, shortcut_roots: set[str]) -> bool:
5     """True only for paths this lakehouse owns, never under a shortcut."""
2     if any(path == root or path.startswith(root.rstrip("/") + "/") for
      root in shortcut_roots):
7         return False
8     return path.startswith(OWNED_PREFIXES)
```

1. **Line 1.** An allowlist of folders this lakehouse owns.
2. **Line 6.** Anything under a shortcut is refused, whatever its name.

Shortcuts save copies. Treat every path under a shortcut as someone else's data.

Recap card

- **What a shortcut is** A folder that points to data in OneLake or external storage, without copying it.
- **Delete the shortcut vs delete inside it** Deleting the shortcut keeps the data. Deleting files inside it deletes them in the target, if you can write there.

- **Shortcut as a table** Create it under Tables, pointing at Delta data.
- **External shortcut caching** OneLake caches read files for 1 to 28 days to cut egress.

Practice

Exercise 1. Multiple choice (level 1)

`Files/partner_feed` is a shortcut to an ADLS Gen2 folder. You delete the shortcut `Files/partner_feed` itself. What happens to the data in ADLS?

- a) It is deleted too.
- b) It stays. Deleting a shortcut deletes only the shortcut object.
- c) It is moved to the lakehouse recycle bin.
- d) It is copied into OneLake before the shortcut goes.

Why?

- r1) Shortcuts do not cascade deletes. The shortcut is an independent object.
- r2) ADLS data can never be deleted from Fabric.

Exercise 2. Multiple choice (level 2)

Your identity has write access to the target storage. You delete `Files/partner_feed/2023/` where `partner_feed` is a shortcut. What happens?

- a) Only the lakehouse view of 2023 is hidden.
- b) The 2023 folder is deleted in the target storage account.
- c) The delete fails: shortcuts are read-only.
- d) The shortcut breaks, but the data stays.

Why?

- r1) Deleting a file or folder within a shortcut deletes it in the target when you have permission there.
- r2) Fabric syncs deletes back every night.

Exercise 3. Fill the gap (level 2)

Complete the rule for where to create a shortcut so the SQL analytics endpoint sees it as a table.

```
1 Create the shortcut under _____(1), pointing at data in _____(2) format.
```

Exercise 4. Write the function (level 3)

Write `safe_to_delete(path: str, shortcut_roots: set[str]) -> bool` for a clean-up job. Return `False` if `path` is a shortcut root or lies under one. Otherwise return `True` only if it starts with `"Files/landing/"` or `"Files/tmp/"`. Treat `"Files/partner"` and `"Files/partner_old/x"` as different folders.

Your code must pass these checks:

- Anything at or under a shortcut root must be refused.
- Paths under Files/landing/ or Files/tmp/ that are not under a shortcut are safe.
- Match whole folder names: Files/partner does not contain Files/partner_old, and Files/landing_old is not owned.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
OWNED_PREFIXES = ...

# True only for owned paths that are not under a shortcut.
def safe_to_delete(
    path: str,
    shortcut_roots: set[str],
) -> bool: ...
```

Starter code:

```python
def safe_to_delete(path, shortcut_roots):
    return path.startswith("Files/")
```

Exercise 5. Architecture design (level 4)

Marketing's campaign data lives in an S3 bucket, updated daily. Your lakehouse needs it for Spark and SQL. Choose the set-up.

Options: An S3 shortcut, A nightly copy pipeline into Files.

1. How the data reaches the lakehouse
2. Where the shortcut sits, given the data is Delta
3. Cross-cloud reads
4. Identity used by the connection

Sources

- Shortcut targets, Tables vs Files behaviour, no cascading deletes, deletes inside a shortcut reach the target with permission, caching 1 to 28 days, delegated identity not passed through. (verified: https://

learn.microsoft.com/en-us/fabric/onelake/onelake-shortcuts (updated 2026-07-13))

- notebookutils.fs.ls(path) lists a directory and notebookutils.fs.rm(path, recurse) deletes; MSSparkUtils was renamed NotebookUtils. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/notebookutils/notebookutils-file-system and .../notebook-utilities)

C3-L2 Mirroring operational databases into OneLake

Concept

Mirroring brings an external database into Fabric as a unit. For sources that store data in their own formats, such as Azure SQL Database, Azure SQL Managed Instance, Azure Cosmos DB, Snowflake or Google BigQuery, Fabric **continuously replicates** the chosen tables into OneLake as Delta tables, in near real time. You get a mirrored database item and a **SQL analytics endpoint** over it, read-only, like a lakehouse's.

Shortcut or mirroring? Microsoft Learn's rule:

- **Shortcut** when you want specific data, already in an open format, in your OneLake view without copying it.
- **Mirroring** when you want an external database or catalog, and the data behind it, in Fabric as a unit. It replicates when the source uses a proprietary format.

Cost. For database mirroring and open mirroring, the background compute that replicates data does not consume capacity. Mirrored storage is free up to **1 TB per capacity unit**: an F64 includes 64 TB of mirroring storage. Queries on the mirrored data (Spark, SQL, Power BI) use capacity as usual.

Limits come per source. For Azure SQL Database, checked September 2026:

- at most 1,000 mirrored tables per database,
- tables whose primary key uses `datetime2(7)`, `datetimeoffset(7)` or `time(7)` cannot be mirrored,
- since April 2025 tables without a primary key can be mirrored, but existing ones are not added automatically, even with "Automatically mirror future tables" turned on.

Status varies too. Most sources are generally available. Some, such as Azure Database for MySQL, Dremio and SharePoint List, were in preview in

September 2026. Check the mirroring overview page before you build on a source.

After you set up a mirror, compare row counts per table between source and mirror (lesson A6-L3). The limits above fail quietly: an ineligible table is simply not there.

Why it matters

A team mirrored an Azure SQL order database and built a dashboard on it. Weeks later, finance noticed that returns were missing. The returns table's primary key included a `datetime2(7)` column, so it had never been mirrored. Nothing errored. A per-table row-count check between source and mirror on the first day would have listed it as absent.

Worked example: weak code

```
1 -- Mirror set up for all tables. Dashboard built on the mirror.
2 SELECT COUNT(*) FROM mirror_orders.dbo.orders;
```

1. **Line 1.** Assumes every table was eligible and replicated.

Worked example: strong code

```
1  -- Run in the source database: tables and row counts
1  SELECT s.name AS schema_name, t.name AS table_name, SUM(p.rows) AS
     source_rows
3  FROM sys.tables AS t
4  JOIN sys.schemas AS s ON s.schema_id = t.schema_id
5  JOIN sys.partitions AS p ON p.object_id = t.object_id AND p.index_id IN
     (0, 1)
6  GROUP BY s.name, t.name;
7
8  -- Run on the mirror's SQL analytics endpoint: which tables arrived
9  SELECT TABLE_SCHEMA, TABLE_NAME
2  FROM INFORMATION_SCHEMA.TABLES
11 WHERE TABLE_TYPE = 'BASE TABLE';
```

1. **Line 2.** The expected list and sizes, from the source.
2. **Line 10.** The actual list in the mirror. Any table missing here was not mirrored.

Mirroring replicates operational databases for free compute and generous storage, within per-source limits. Reconcile the table list after setup.

Recap card

- **Shortcut or mirroring?** Shortcut for open-format data in storage. Mirroring for databases and catalogs, replicated when proprietary.

- **Mirroring cost** Replication compute is free. Storage is free up to 1 TB per CU. Queries use capacity.
- **Azure SQL Database mirroring limits** Up to 1,000 tables. Primary keys on datetime2(7), datetimeoffset(7) or time(7) block a table.
- **After setting up a mirror** Reconcile table lists and row counts with the source.

Practice

Exercise 1. Multiple choice (level 1)

You need an Azure SQL Database's tables in Fabric, kept up to date, for reporting. Shortcut or mirroring?

- a) Shortcut: shortcuts connect to Azure SQL Database.
- b) Mirroring: it replicates the database into OneLake continuously.
- c) Neither: export CSV files nightly.
- d) A shortcut under Tables.

Why?

- r1) Azure SQL stores data in a proprietary format, so Fabric must replicate it, which is what mirroring does.
- r2) Mirroring is always cheaper than shortcuts.

Exercise 2. Predict the output (level 2)

Using the documented rule of 1 TB of free mirroring storage per capacity unit, what does this print?

```
capacity_units = 32
mirrored_tb = 45
free_tb = capacity_units * 1
print(free_tb, max(0, mirrored_tb - free_tb))
```

Exercise 3. Fill the gap (level 2)

List the base tables that arrived in the mirror, on its SQL analytics endpoint.

```
SELECT TABLE_SCHEMA, TABLE_NAME
FROM _____(1).TABLES
WHERE TABLE_TYPE = '_____(2)';
```

Exercise 4. Write the function (level 3)

Write `missing_from_mirror(source: dict[str, int], mirror: dict[str, int]) -> list[str]` for a reconciliation job. Both map `schema.table` to a row count.

Return, sorted, the tables that are absent from the mirror, or whose mirror count differs from the source count.

Your code must pass these checks:

- Tables missing from the mirror must be listed.
- Tables whose counts differ must be listed too.
- Return a sorted list, and ignore tables that exist only in the mirror.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Source tables that are absent from the mirror or differ in row count.
def missing_from_mirror(
    source: dict[str, int],
    mirror: dict[str, int],
) -> list[str]: ...
```

Starter code:

```python
def missing_from_mirror(source, mirror):
    return [t for t in source if t not in mirror]
```

Exercise 5. Architecture design (level 4)

Plan mirroring for an Azure SQL order database with 1,400 tables. Some primary keys use `datetime2(7)`.

Options: The tables reporting needs, under the 1,000-table limit, All 1,400.

1. Which tables to mirror
2. Tables with datetime2(7) primary keys
3. After setup

Sources

- Shortcuts vs mirroring guidance, and mirroring replicates when the source stores data in a proprietary format. (verified: https://learn.microsoft.com/en-us/fabric/onelake/unify-data (updated 2026-07-15))
- Replication compute is free and does not consume capacity; mirroring storage is free at 1 TB per CU; MySQL, Dremio and SharePoint List sources are in preview; BigQuery is GA. (verified: https://learn.microsoft.com/en-us/fabric/mirroring/overview (updated 2026-08-28))
- Azure SQL Database mirroring: 1,000 tables maximum, datetime2(7)/datetimeoffset(7)/time(7) primary keys unsupported, tables without primary keys supported since April 2025 but not added automatically. (verified: https://learn.microsoft.com/en-us/fabric/mirroring/azure-sql-database-limitations (updated 2026-02-26))

C3-L3 Mirroring Azure Databricks Unity Catalog into Fabric

Concept

Your Track A pipelines write gold tables into Unity Catalog. **Mirroring Azure Databricks Unity Catalog** makes them readable in Fabric without copying them. Unlike database mirroring, **no data moves**. Fabric mirrors only the catalog structure (catalogs, schemas, tables) and reads the underlying Delta files through shortcuts.

You get a mirrored Azure Databricks item with a SQL analytics endpoint, and you can build Power BI reports on it in Direct Lake mode. By default, **Automatically sync future catalog changes for the selected schema** is on, so new tables appear. Propagation takes from seconds to several minutes.

Three rules decide whether it works and whether it is safe (Microsoft Learn, checked September 2026):

1. Some table types do not mirror: tables with row filters or column masks (RLS/CLM policies), Lakehouse federated tables, Delta Sharing tables, streaming tables, views and materialized views. The ADLS Gen2 account behind the Databricks workspace must also be reachable from Fabric.

2. Unity Catalog permissions do not carry over. Grants on catalogs, schemas and tables in Databricks do not apply in Fabric. You rebuild access with Fabric's model: workspace roles, item permissions, SQL permissions, OneLake security.

3. One credential reads everything. The credential used to create the connection is used for all data queries. Unity Catalog policies such as row filters, column masks or ABAC are not enforced at the storage layer, so they would not apply to direct storage access anyway.

Put together: mirror tables that are safe to share at the level you grant in Fabric. For data that needs row-level rules, publish a gold table without the policy that already contains only what the audience may see, or apply the rules again in Fabric.

Why it matters

A team mirrored their gold schema so that regional managers could build Power BI reports. In Databricks, a row filter limited each manager to their region (lesson A4-L3). The filtered table did not appear in Fabric at all, so the

team created an unfiltered copy for the mirror, with a connection that could read every row. Every manager with Viewer access now saw every region. The rules had lived in Unity Catalog, and nothing had been rebuilt in Fabric.

Worked example: weak code

```
1 Mirror: catalog gold_prod, schema sales, all tables, auto-sync on
2 Connection: service principal with SELECT on the whole catalog
3 Fabric workspace: regional managers as Viewers
3 Assumption: Unity Catalog row filters still apply
```

1. **Line 1.** Tables with row filters are not mirrored, and any new table appears automatically.
2. **Line 2.** This credential reads every mirrored table for every user.
3. **Line 4.** Unity Catalog permissions and policies do not carry over to Fabric.

Worked example: strong code

```
1 Mirror: catalog gold_prod, schema sales_shareable (tables approved for all
    managers)
2 Connection: service principal with SELECT on sales_shareable only
3 Regional detail: rebuilt in Fabric with row-level security on the semantic
    model
4 Check: row counts per mirrored table match Databricks after each release
```

1. **Line 1.** A schema of tables that are safe for the whole Fabric audience.
2. **Line 2.** Least privilege on the one credential that reads everything.
3. **Line 3.** Access rules recreated where the users are.

Mirroring Unity Catalog copies no data and no permissions. Decide what is shareable, scope the connection, and rebuild access rules in Fabric.

Recap card

- **What mirroring Unity Catalog moves** Only catalog metadata. Data is read in place through shortcuts.
- **Unity Catalog tables that do not mirror** Tables with row filters or column masks, federated, Delta Sharing and streaming tables, views and materialized views.
- **Unity Catalog permissions in Fabric** Not carried over. Rebuild access with Fabric's permission model.
- **Which identity reads mirrored data?** The connection's credential, for all queries. Scope it tightly.

Practice

Exercise 1. Multiple choice (level 1)

When Fabric mirrors an Azure Databricks Unity Catalog, what moves into OneLake?

- a) A full copy of every table's data
- b) Only the catalog structure. Data is read through shortcuts.
- c) Nightly snapshots of changed partitions
- d) Nothing. Fabric queries Databricks SQL warehouses.

Why?

- r1) Unity Catalog tables are already open Delta files, so Fabric mirrors metadata and reads the files in place.
- r2) Copying is blocked by Databricks licensing.

Exercise 2. Multiple choice (level 2)

A gold table in Unity Catalog has a row filter. You select its schema for mirroring. What happens to that table?

- a) It is mirrored with the row filter enforced in Fabric.
- b) It is mirrored without the filter.
- c) It is not mirrored: tables with RLS or CLM policies are unsupported.
- d) It is mirrored as a view.

Why?

- r1) The limitations page lists tables with RLS/CLM policies as unsupported.
- r2) Row filters make tables read-only.

Exercise 3. Put in order (level 2)

Order the steps to publish Databricks gold tables to Fabric users safely.

- Create a connection identity with SELECT on that schema only
- Compare table lists and row counts with Databricks
- Create the mirrored Azure Databricks item for that schema
- Choose tables safe for the whole Fabric audience, in a dedicated schema
- Grant Fabric access with workspace roles, SQL permissions or model RLS

Exercise 4. Write the function (level 3)

Write `mirror_eligible(tables: list[dict[str, object]]) -> list[str]`. Each dict has `name` (str), `table_type` (str) and `has_row_filter_or_mask` (bool).

Return, sorted, the names that can be mirrored: `table_type` must be `"MANAGED"` or `"EXTERNAL"`, and the table must not have a row filter or column mask. Streaming tables, views, materialized views, federated and Delta Sharing tables arrive with other `table_type` values.

Your code must pass these checks:

- Only MANAGED and EXTERNAL tables qualify, returned sorted.
- Tables with row filters or column masks are not mirrored.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
MIRRORABLE_TYPES = ...

# Unity Catalog tables that Fabric can mirror.
def mirror_eligible(
    tables: list[dict[str, object]],
) -> list[str]: ...
```

Starter code:

```python
def mirror_eligible(tables):
    return [t["name"] for t in tables]
```

Exercise 5. Architecture design (level 4)

Regional managers must each see only their region's sales in Power BI. The data is a Unity Catalog gold table with a row filter. Choose the design.

Options: An unfiltered copy of the table, for all managers, An unfiltered table, with row-level security rebuilt in the Fabric semantic model.

1. What Fabric mirrors
2. Connection identity
3. Manager access to the mirror's SQL endpoint

Sources

- Only the Azure Databricks catalog structure is mirrored and data is accessed through shortcuts, with no data movement; auto-sync of future catalog changes is on by default; propagation takes seconds to several minutes. (verified: https://learn.microsoft.com/en-us/fabric/mirroring/azure-databricks (updated 2025-10-24))
- Unsupported table types: RLS/CLM tables, Lakehouse federated tables, Delta Sharing tables, streaming tables, views and materialized views; the Databricks ADLS account must be accessible to Fabric. (verified: https://

learn.microsoft.com/en-us/fabric/mirroring/azure-databricks-limitations (updated 2025-12-19))

- Unity Catalog permissions are not mirrored; the connection credential is used for all data queries; UC policies are not enforced at the storage layer. (verified: https://learn.microsoft.com/en-us/fabric/mirroring/azure-databricks-security (updated 2026-05-01))
- Unity Catalog information_schema.tables reports table_type values including MANAGED, EXTERNAL, VIEW, MATERIALIZED_VIEW, STREAMING_TABLE and FOREIGN. (verified: https://docs.databricks.com/aws/en/sql/language-manual/information-schema/tables)

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C3-L1 OneLake shortcuts, and the delete that goes through

C3-L1 exercise 1

Hints. (1) Is the shortcut the data or a pointer? (2) What does removing a pointer do? (3) Contrast it with deleting inside the pointer.

Answer. b) It stays. Deleting a shortcut deletes only the shortcut object. Because: r1) Shortcuts do not cascade deletes. The shortcut is an independent object.

The shortcut goes, the data stays. Deleting a file or folder inside the shortcut is different.

Common wrong answers.

- `a`: Removing the shortcut removes only the link. Deleting files inside it is what reaches the target.
- `c`: There is no copy to recycle. The data was never in OneLake.
- `d`: Nothing is copied when you delete a shortcut.
- `b|r2`: Right answer, wrong reason. Deleting files inside a shortcut can delete ADLS data.

C3-L1 exercise 2

Hints. (1) Where do the files inside a shortcut live? (2) In the target. (3) Your permissions there decide.

Answer. b) The 2023 folder is deleted in the target storage account. Because: r1) Deleting a file or folder within a shortcut deletes it in the target when you have permission there.

Inside a shortcut you are working on the target. Deletes go through.

Common wrong answers.

- `a`: There is no hidden view. The folder is the target folder.
- `c`: Shortcuts are not read-only when your identity can write to the target.
- `d`: The shortcut still works. It just points at data that no longer includes 2023.

- `b|r2`: Right answer, wrong reason. The delete happens immediately, not on a schedule.

C3-L1 exercise 3

Hints. (1) Which area holds managed tables? (2) Which format does the endpoint read? (3) Tables and Delta.

Answer.

```
Create the shortcut under Tables, pointing at data in Delta format.
```

A shortcut under Tables that points at Delta data is recognised as a table.

Common wrong answers.

- g1 = `Files`: Shortcuts under Files can hold any format, but they do not become tables.

C3-L1 exercise 4

Hints. (1) Check shortcuts first and refuse early. (2) Compare against root + "/" so partner does not match partner_old. (3) Allowlist owned prefixes with the trailing slash.

Answer.

```python
OWNED_PREFIXES = ("Files/landing/", "Files/tmp/")


def safe_to_delete(path: str, shortcut_roots: set[str]) -> bool:
    """True only for owned paths that are not under a shortcut.

    Args:
        path: Lakehouse-relative path, such as Files/landing/w14.csv.
        shortcut_roots: Paths of every shortcut in the lakehouse.

    Returns:
        Whether a clean-up job may delete the path.
    """
    for root in shortcut_roots:
        base = root.rstrip("/")
        if path == base or path.startswith(base + "/"):
            return False
    return path.startswith(OWNED_PREFIXES)
```

Refusing anything under a shortcut, and allowing only named folders, makes a clean-up job unable to reach someone else's data.

C3-L1 exercise 5

Hints. (1) Copy or point? (2) Where do shortcuts become tables? (3) What stops a delete from going through?

Answer.

- A shortcut avoids a second copy and a pipeline to maintain.
- Under Tables, Delta data becomes a table for SQL and Direct Lake.
- Caching cuts egress costs for repeated reads from S3.
- A read-only identity means no delete inside the shortcut can reach the bucket.

Point, place it where it becomes a table, cache cross-cloud reads, and connect read-only.

Common wrong answers.

- access = `copy`: A copy doubles storage and needs a pipeline and monitoring. A shortcut reads in place.
- writes = `owner`: With write access, deleting inside the shortcut deletes marketing's data.
- location = `files`: Under Files it will not be recognised as a table.

C3-L2 Mirroring operational databases into OneLake

C3-L2 exercise 1

Hints. (1) What do shortcuts point at? (2) Is an Azure SQL database a folder of Delta files? (3) It must be replicated.

Answer. b) Mirroring: it replicates the database into OneLake continuously. Because: r1) Azure SQL stores data in a proprietary format, so Fabric must replicate it, which is what mirroring does.

Databases in proprietary formats come in through mirroring. Open-format data in storage comes in through shortcuts.

Common wrong answers.

- `a`: Shortcuts point at storage (OneLake, ADLS, S3 and similar), not at database engines.
- `c`: A nightly export is a hand-built copy with lag and maintenance. Mirroring does it continuously.
- `d`: There is no storage folder of Delta files to point at. The database must be replicated.
- `b|r2`: Right answer, wrong reason. Shortcuts copy nothing. Mirroring is needed because the source is a database.

C3-L2 exercise 2

Hints. (1) How many CUs does the capacity have? (2) 1 TB free per CU. (3) 45 - 32.

Answer.

```
32 13
```

32 TB are free on an F32. The remaining 13 TB of mirrored storage is billed as OneLake storage.

Common wrong answers.

- `64 0`: The example on Learn uses F64. This capacity has 32 CUs, so 32 TB are free.

C3-L2 exercise 3

Hints. (1) The standard catalog views schema. (2) Tables, not views. (3) `INFORMATION_SCHEMA` and `BASE TABLE`.

Answer.

```sql
SELECT TABLE_SCHEMA, TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE';
```

`INFORMATION_SCHEMA.TABLES` lists the tables the endpoint exposes. Compare it with the source list.

Common wrong answers.

- g2 = `VIEW`: Views are not mirrored tables. Filter on BASE TABLE.

C3-L2 exercise 4

Hints. (1) Loop over the source tables. (2) mirror.get(table) is None when absent, so one comparison covers both cases. (3) Sort the result.

Answer.

```python
def missing_from_mirror(source: dict[str, int], mirror: dict[str, int]) -> list[str]:
    """Source tables that are absent from the mirror or differ in row count.

    Args:
        source: schema.table to row count in the source database.
        mirror: schema.table to row count on the mirror's endpoint.

    Returns:
        Sorted table names that need attention.
    """
    return sorted(t for t, n in source.items() if mirror.get(t) != n)
```

One comparison, `mirror.get(t) != n`, catches both missing tables and partial replication.

C3-L2 exercise 5

Hints. (1) Is there a table limit? (2) Which key types block mirroring? (3) How would you notice a missing table?

Answer.

- Mirror what reporting needs, within the 1,000-table limit.
- Tables with datetime2(7) keys cannot be mirrored. Plan for them explicitly.
- Reconcile daily, because ineligible tables fail quietly.

Plan within the documented limits, then verify with data, not with status pages.

Common wrong answers.

- scope = `all`: Azure SQL Database mirroring supports at most 1,000 tables.
- keys = `ignore`: Those tables are not eligible. They will simply be missing.
- check = `trust`: A table that was never eligible does not show as failing. Compare lists and counts.

C3-L3 Mirroring Azure Databricks Unity Catalog into Fabric

C3-L3 exercise 1

Hints. (1) What format are Unity Catalog managed tables stored in? (2) Can Fabric read that format directly? (3) Metadata plus shortcuts.

Answer. b) Only the catalog structure. Data is read through shortcuts. Because: r1) Unity Catalog tables are already open Delta files, so Fabric mirrors metadata and reads the files in place.

Only the catalog structure is mirrored. The data stays in the Databricks storage account.

Common wrong answers.

- `a`: That is database mirroring for proprietary sources. Unity Catalog mirroring has no data movement.
- `c`: There are no snapshots. Fabric reads the current Delta files through shortcuts.

- `d`: Fabric engines read the Delta files directly. No Databricks compute runs the queries.
- `b|r2`: Right answer, wrong reason. It is about format: Delta can be read in place.

C3-L3 exercise 2

Hints. (1) Check the list of unsupported table types. (2) RLS/CLM policies. (3) The table is left out.

Answer. c) It is not mirrored: tables with RLS or CLM policies are unsupported. Because: r1) The limitations page lists tables with RLS/CLM policies as unsupported.

Filtered and masked tables are excluded. Decide what to publish for Fabric users instead.

Common wrong answers.

- `a`: Unity Catalog policies are not carried into Fabric, and such tables are not supported.
- `b`: Fabric does not quietly drop the filter. It does not mirror the table.
- `d`: Views are themselves unsupported for this mirroring.
- `c|r2`: Right answer, wrong reason. The table type is unsupported, not read-only.

C3-L3 exercise 3

Hints. (1) Scope before you connect. (2) The connection's rights cap what is readable. (3) Access, then verify.

Answer.

1. Choose tables safe for the whole Fabric audience, in a dedicated schema
2. Create a connection identity with SELECT on that schema only
3. Create the mirrored Azure Databricks item for that schema
4. Grant Fabric access with workspace roles, SQL permissions or model RLS
5. Compare table lists and row counts with Databricks

Scope, least-privilege connection, mirror, Fabric access, then a data check.

Common wrong answers.

- Mirroring first exposes whatever the connection can read. Decide the scope first.
- Set up access before users arrive. Verification then confirms what they will see.

C3-L3 exercise 4

Hints. (1) Allow a fixed set of table types rather than excluding a list. (2) Also exclude tables with policies. (3) Sort the names.

Answer.

```python
MIRRORABLE_TYPES = frozenset({"MANAGED", "EXTERNAL"})


def mirror_eligible(tables: list[dict[str, object]]) -> list[str]:
    """Unity Catalog tables that Fabric can mirror.

    Args:
        tables: Dicts with name, table_type and has_row_filter_or_mask.

    Returns:
        Sorted names of eligible tables.
    """
    return sorted(
        str(t["name"])
        for t in tables
        if t["table_type"] in MIRRORABLE_TYPES and not
    t["has_row_filter_or_mask"]
    )
```

An allowlist of table types plus the policy check predicts which tables will appear, so a missing table is expected rather than a surprise.

C3-L3 exercise 5

Hints. (1) Where do Unity Catalog rules stop applying? (2) Whose credential reads the data? (3) Where does the RLS live?

Answer.

- The filtered table cannot be mirrored, so rebuild the regional rule where users consume the data.
- Scope the connection, because it reads every mirrored table for everyone.
- Keep managers on the report. Direct endpoint access would bypass the model's RLS.

Rebuild the rule where users are, scope the credential, and keep users on the governed path.

Common wrong answers.

- source = `unfiltered_all`: Every manager would see every region. The rule must be rebuilt in Fabric.
- connection = `broad`: The connection's credential serves all queries. Grant it only what the mirror needs.

- users = `viewer`: Viewers can query the endpoint with T-SQL and read rows the model's RLS would hide.